

1. Introduzione

1.1 Background e storia del progetto

FleetManager nasce come progetto accademico per il corso di Ingegneria del Software A.A. 2025/26 (Prof. Angelo Gargantini, Dott.ssa Silvia Bonfanti), con l'obiettivo di applicare in modo pratico le metodologie di analisi, progettazione e gestione studiate a lezione.

Il progetto si colloca all'interno del percorso di valutazione del corso e costituisce parte integrante dell'esame finale. Tutta la documentazione, i diagrammi e il codice sorgente vengono mantenuti in un repository GitHub condiviso con i docenti (garganti, silviabonfanti) e aggiornato progressivamente nel corso dello sviluppo.

FleetManager nasce dall'esigenza di disporre di un sistema software per la gestione centralizzata di flotte aziendali, che consenta di amministrare i veicoli, gestire le prenotazioni dei dipendenti e monitorare scadenze e manutenzioni.

Il progetto adotta le tecnologie e i vincoli richiesti dal corso:

- Java 17 come linguaggio di programmazione;
- Eclipse come IDE;
- Maven per la gestione delle dipendenze;
- Papyrus UML per la modellazione;
- JUnit 5 per i test;
- Database H2 embedded;
- GitHub per la collaborazione e il versionamento.

Come previsto dalle linee guida, il Project Plan viene consegnato circa un mese prima dell'esame, mentre il progetto completo deve essere finalizzato cinque giorni prima della presentazione.

1.2 Obiettivi del progetto

- Realizzare un sistema gestionale completo per flotte aziendali che permetta:
 - la gestione dell'anagrafica dei veicoli,
 - la prenotazione e restituzione da parte dei dipendenti,
 - la pianificazione delle manutenzioni e il monitoraggio delle scadenze (bollo, assicurazione, revisione).
- Applicare un processo di sviluppo Agile iterativo con sprint brevi e consegne incrementali.
- Utilizzare in modo integrato UML → Java → JUnit, garantendo tracciabilità tra requisiti, design e test.
- Dimostrare la padronanza degli strumenti professionali previsti (Eclipse, Maven, GitHub, Papyrus).
- Produrre documentazione completa e conforme al modello Van Vliet (14 punti).

1.3 Risultati attesi

- Repository GitHub con la struttura richiesta (documenti/ + codice/) e uso effettivo di issue, branch e pull request.
- Progetto Eclipse/Maven funzionante, privo di riferimenti assoluti, con test automatizzati JUnit.
- Progetto Papyrus UML contenente i diagrammi richiesti (use case, class, state, sequence, activity, component, package).
- Documentazione completa in PDF: Project Plan, Gestione progetto, Requisiti, Design, Testing, Maintenance.
- Presentazione PowerPoint per l'esame orale e demo eseguibile del sistema.

1.4 Nomi delle persone responsabili

Il team di sviluppo è composto da due membri, che collaborano in tutte le fasi progettuali, condividendo responsabilità e versionamento:

- Francesco Basis: 1092076 – f.basis@studenti.unibg.it
- Pietro Plati: 1091949 – p.plati@studenti.unibg.it

1.5 Sintesi del progetto

FleetManager è una soluzione software progettata per semplificare la gestione operativa di una flotta aziendale, automatizzando prenotazioni, controlli di stato e promemoria di manutenzione. Il progetto combina modellazione UML, programmazione Java, testing automatizzato e versionamento collaborativo GitHub per produrre un sistema completo, funzionante e documentato, in linea con i principi del capitolo 2 di Van Vliet e con le richieste ufficiali del corso.

2. Modello di processo

2.1 Scelta del modello

Per FleetManager è stato scelto un modello di processo Agile iterativo–incrementale, ispirato al framework Scrum ma adattato alle dimensioni ridotte del team (2 membri: Pietro Plati e Francesco Basis).

L'obiettivo è garantire flessibilità, feedback continuo e integrazione progressiva dei moduli, riducendo i rischi e mantenendo una documentazione sempre aggiornata.

Il processo Agile è preferito rispetto al modello a cascata perché:

- i requisiti possono evolversi man mano che emergono nuove necessità o vincoli tecnici;
- la frequenza delle revisioni intermedie consente di verificare il raggiungimento delle pietre miliari e la qualità del prodotto;
- permette di testare funzionalità complete a ogni incremento, riducendo il rischio di integrazione finale.

2.2 Struttura del processo

Il ciclo di sviluppo prevede sprint brevi, ciascuno con obiettivi chiari, deliverable verificabili e test JUnit associati.

Ogni sprint include le fasi fondamentali del ciclo di vita software:

1. Pianificazione (definizione obiettivi e requisiti incrementali)
2. Progettazione UML (aggiornamento dei diagrammi in Papyrus)
3. Implementazione e integrazione del codice Java/Maven
4. Verifica e test (unit test JUnit e review GitHub)
5. Retrospettiva e aggiornamento documentazione

2.3 Pietre Miliari

Milestone	Descrizione	Deliverable
M0 – Setup	Creazione repository GitHub, configurazione progetto Maven e baseline Papyrus	Repo iniziale, .project, pom.xml
M1 – Modellazione base	Diagrammi Use case e Class Diagram principali	File .uml Papyrus, documento requisiti
M2 – Funzionalità core	Implementazione CRUD veicoli e utenti	Codice Java funzionante, test unitari
M3 – Prenotazioni e stati veicolo	Logica prenotazioni e State Machine Diagram	Service Java, diagrammi aggiornati
M4 – Scadenze e notifiche	Gestione automatica scadenze, pattern Observed	Codice Java, test JUnit, log 4j
M5 – Refactoring e metriche	Misure qualità, complessità, revisione finale	Report SonarLint/PMD, documenti finale
M6 – Consegna e demo	Consegna finale e presentazione orale	Repository completo + PowerPoint finale

2.4 Criteri di verifica delle pietre miliari

Ogni milestone è considerata raggiunta se:

- tutti i test JUnit associati superano la verifica;
- i diagrammi UML sono aggiornati nel repository;
- è presente un commit firmato da entrambi i membri;
- la documentazione è coerente con l'implementazione.

2.5 Percorso critico

Il percorso critico del progetto coincide con la modellazione iniziale e la prima integrazione funzionale (M1–M3). Eventuali ritardi in queste fasi comportano slittamenti su tutte le successive attività (testing, metriche, refactoring).

Per questo motivo:

- i diagrammi UML saranno sviluppati in modo iterativo già dallo Sprint 0;
- i test verranno creati in parallelo all'implementazione;
- la documentazione sarà aggiornata incrementando la stessa base GitHub, evitando accumulo di lavoro nelle fasi finali.

2.6 Pianificazione temporale indicativa

(ancora da stabilire)

2.7 Controllo del processo

- Revisione tecnica a fine sprint tra i due membri del team.
- Retrospettiva breve per identificare problemi e miglioramenti.
- Aggiornamento del Project Plan in caso di deviazioni significative.

3. Organizzazione del progetto

3.1 Relazioni organizzative e contesto

Il progetto FleetManager è sviluppato nell'ambito del corso di Ingegneria del Software A.A. 2025/26, sotto la supervisione del Prof. Angelo Gargantini e della Dott.ssa Silvia Bonfanti, che rappresentano l'organizzazione madre e l'ente valutatore.

Il team di progetto è composto da Pietro Plati e Francesco Basis, che operano come organizzazione utente e di sviluppo, gestendo in autonomia analisi, progettazione, implementazione, testing e documentazione.

Il progetto ha relazioni con:

- Docenti del corso, che forniscono linee guida, feedback e validazione del project plan;
- Repository GitHub, utilizzato come piattaforma di collaborazione, controllo versione e verifica dei progressi;
- Strumenti software di supporto (Eclipse, Papyrus, Maven, SonarQube, H2 Database), che rappresentano le risorse operative per lo sviluppo e la validazione.

3.2 Coinvolgimento degli utenti

I potenziali utenti del sistema FleetManager sono:

- il Fleet Manager, responsabile della gestione della flotta aziendale (prenotazioni, manutenzioni, scadenze);
- i Driver/Dipendenti, che prenotano e utilizzano i veicoli.

Durante lo sviluppo, questi ruoli vengono simulati dai membri del team stesso, che ne analizzano i requisiti funzionali e ne verificano il corretto comportamento attraverso casi d'uso e test.

Le informazioni e i servizi forniti dagli "utenti simulati" includono:

- l'elenco dei requisiti funzionali;
- i flussi di utilizzo e le regole di business
- i dati di test e di simulazione per la validazione del software.

3.3 Struttura del team

Il team è autonomo e cooperativo, composto da due membri con ruoli definiti ma complementari.

La comunicazione avviene in modo continuo tramite:

- GitHub Issues e Pull Request (per assegnare e tracciare attività);
- Canale Discord privato per discussione tecnica e sincronizzazione giornaliera;
- Meeting settimanali per verifica milestone e retrospettiva.

Entrambi i membri contribuiscono in modo attivo a:

- definizione e affinamento dei requisiti;
- implementazione delle funzionalità principali;
- revisione incrociata del codice e dei documenti.

3.4 Formazione e competenze

Il team ha svolto una formazione preliminare su:

- uso di Papyrus UML per la modellazione;
- gestione di progetti Maven e configurazione delle dipendenze;
- test automatizzati con JUnit 5;
- utilizzo di Git e GitHub per versionamento e code review.

3.5 Struttura organizzativa e flusso decisionale

Le decisioni principali (approvazione requisiti, milestone, merge su main) vengono prese congiuntamente da entrambi i membri.

Il processo decisionale segue tre livelli:

1. Pianificazione (definizione obiettivi sprint e assegnazione attività)
2. Esecuzione (sviluppo e testing autonomo su branch dedicati)
3. Revisione (code review e validazione milestone da parte di entrambi)

3.6 Sintesi

L'organizzazione del progetto FleetManager è quindi strutturata su un modello collaborativo a due membri, con ruoli bilanciati e responsabilità chiare.

L'approccio è coerente con il principio espresso da Van Vliet, secondo cui l'organizzazione del team e la definizione dei ruoli sono fondamentali per garantire efficienza, comunicazione e qualità nel ciclo di sviluppo.

4. Standard, linee guida e procedure

4.1 Disciplina di lavoro

Il progetto FleetManager adotta una forte disciplina di lavoro, basata su procedure condivise e verificabili, in modo che ciascun membro del team segua gli stessi standard, convenzioni e modalità operative.

Tutti gli standard sono documentati nel repository GitHub e applicati quotidianamente tramite strumenti automatici (Maven, Git, SonarQube, Papyrus, JUnit).

L'obiettivo è assicurare uniformità, tracciabilità e qualità costante tra codice, documentazione e diagrammi UML, come richiesto dalle linee guida del corso di Ingegneria del Software e dai principi del Van Vliet.

4.2 Standard di codifica

- Linguaggio: Java 17, con le convenzioni ufficiali Oracle per la formattazione e la nomenclatura (camelCase per variabili e metodi, PascalCase per classi, UPPER_CASE per costanti).
- Struttura dei package:
 - (ancora da definire)
- Commenti e documentazione interna:
 - Ogni classe e metodo pubblico deve contenere JavaDoc.
 - Tutte le variabili significative devono avere un commento esplicativo.
- Controllo di qualità del codice:
 - Analisi statica con SonarQube.

4.3 Standard per la documentazione

Struttura

Tutta la documentazione è salvata nella cartella /documenti/ del repository, organizzata nei seguenti file PDF/Markdown:

1. (da aggiungere passo passo)

Regole

- Tutti i documenti devono essere versionati su GitHub e aggiornati a ogni sprint.
- Ogni modifica sostanziale deve essere accompagnata da un commit dedicato e da un commento descrittivo.
- Le versioni principali vengono contrassegnate da tag semantici (v1.0, v1.1, ecc.).
- La qualità della documentazione viene valutata secondo tre criteri:
 1. Completezza – tutti i punti richiesti dal piano Van Vliet sono coperti.
 2. Coerenza – i diagrammi e il codice riflettono i requisiti aggiornati.
 3. Aggiornamento – il documento viene aggiornato alla chiusura di ogni sprint.

4.4 Standard di modellazione UML

- Tutti i diagrammi UML sono creati e mantenuti con Papyrus (formato .uml e .di).
- Tipi di diagrammi obbligatori:
 - Use Case Diagram – descrizione dei requisiti funzionali
 - Class Diagram – struttura delle entità principali
 - State Machine Diagram – stati del veicolo
 - Sequence Diagram – interazioni tra attori e sistema
 - Activity Diagram – flusso prenotazione e restituzione
 - Component e Package Diagram – organizzazione architetturale
- Ogni modifica del codice che impatta la logica del dominio richiede sincronizzazione del diagramma UML nel commit.

4.5 Procedure operative

- Controllo di versione: ogni attività è tracciata su GitHub tramite branch dedicato e pull request con review obbligatoria.
- Gestione delle dipendenze: centralizzata tramite Maven (pom.xml), con almeno una dipendenza esterna (Log4j2 per logging).
- Build e test: eseguiti tramite mvn clean test.
- Rilascio e validazione: ogni milestone (da M0 a M6) è verificata attraverso una build funzionante e test superati.
- Gestione configurazione: GitHub funge da sistema CM; le issue e i PR documentano le modifiche.

4.6 Strumenti di supporto

Strumento	Funzione
Eclipse IDE	Sviluppo e debug del codice java
Papyrus UML	Modellazione e generazione parziale del codice
Maven	Gestione build e dipendenze
JUnit 5	Testing automatico
Log4j2	Logging di sistema
H2 Database	Database embedded
GitHub	Versionamento, issue tracking e collaborazione
SonarQube	Analisi statica e qualità del codice

4.7 Aggiornamento e controllo

L'aggiornamento degli standard e della documentazione è continuo:

- ogni fine sprint prevede la revisione del codice, dei diagrammi e dei documenti;
- eventuali modifiche agli standard devono essere approvate da entrambi i membri e registrate nel file changelog.md;
- il rispetto degli standard è verificato tramite:
 - linting automatico nella build Maven;
 - code review reciproca;
 - confronto tra documentazione e codice in fase di milestone.

4.8 Sintesi

Gli standard e le procedure di FleetManager garantiscono coerenza, qualità e aggiornamento costante del progetto. Come indicato da Van Vliet, “una forte disciplina di lavoro e accordi chiari sulla documentazione” sono essenziali per il successo di un progetto software: FleetManager traduce questo principio in pratiche operative concrete, integrate nel flusso di lavoro quotidiano.

5. Attività di gestione

5.1 Obiettivi di gestione

Le attività di gestione del progetto FleetManager hanno come scopo principale il monitoraggio costante dello stato di avanzamento e la verifica del rispetto dei vincoli di tempo, qualità e ambito.

Seguendo quanto indicato dal Van Vliet, il team deve:

- produrre relazioni periodiche sull'andamento del lavoro e sull'evoluzione delle milestone;
- bilanciare requisiti, tempi e risorse per garantire la consegna nei termini previsti;
- rilevare tempestivamente eventuali deviazioni e definire azioni correttive.

5.2 Struttura di controllo

Poiché il team è composto da due persone, la gestione assume una forma collaborativa e paritaria. Ogni settimana viene condotto un incontro di revisione interna in cui si analizzano:

- i progressi rispetto alla milestone in corso,
- eventuali problemi tecnici o di pianificazione,
- il tempo effettivamente impiegato,
- i test eseguiti e i risultati ottenuti.

Tutti i dati di avanzamento sono registrati su GitHub Project Board (colonne To Do, In Progress, In Review, Done), che funge da strumento di controllo visivo del progresso.

5.3 Metriche di avanzamento

L'avanzamento del progetto viene monitorato tramite:

- Numero di issue chiuse rispetto al totale previsto per lo sprint;
- Numero di commit e PR integrati nella branch principale;
- Percentuale di test JUnit superati;
- Copertura del codice ≥ 70 %;
- Completezza della documentazione aggiornata a fine sprint.

Ogni sprint si conclude con una breve retrospettiva, in cui si valuta se gli obiettivi sono stati raggiunti e si aggiornano le priorità del backlog.

5.4 Relazioni periodiche

A fine di ogni sprint, viene redatto un report sintetico in formato Markdown (/documenti/report_sprintX.md) contenente:

1. Attività completate e in corso;
2. Stato dei test e qualità del codice;
3. Milestone raggiunte;
4. Eventuali rischi o ritardi;
5. Decisioni e miglioramenti pianificati.

5.5 Bilanciamento requisiti – tempi – costi

Il progetto viene gestito secondo il principio del triangolo di progetto:

- Requisiti: priorità alle funzionalità fondamentali (gestione veicoli, prenotazioni, scadenze).
- Tempo: rispetto rigoroso delle scadenze di sprint e milestone.
- Costi: misurati in termini di ore /uomo, stimati in circa 35–40 ore per persona.

Quando emerge un conflitto tra tempo e ambito, viene adottata la seguente politica:

- non ridurre la qualità, ma ridimensionare temporaneamente la quantità di funzionalità non critiche;
- aggiornare la pianificazione, notificando la variazione nel report dello sprint.

5.6 Indicatori di successo

Un'attività di gestione è considerata efficace se:

- tutte le milestone vengono raggiunte entro la data prevista;
- non si verificano regressioni funzionali nei test;
- la documentazione è coerente con il codice;
- i membri del team mantengono un flusso di commit costante e coordinato.

5.7 Sintesi

Le attività di gestione del progetto FleetManager assicurano controllo continuo, trasparenza e bilanciamento tra obiettivi e risorse. In conformità al Van Vliet, il team garantisce relazioni periodiche sullo stato di avanzamento e mantiene un equilibrio dinamico tra requisiti, tempi e costi, condizione indispensabile per la riuscita del progetto.

6. Rischi

6.1 Identificazione precoce dei rischi

Ogni progetto di sviluppo software comporta inevitabilmente delle incertezze: errori tecnici, ritardi, mancanza di risorse o problemi organizzativi.

Seguendo il principio espresso da Van Vliet — “i rischi devono essere diagnosticati precocemente e devono essere previste misure per affrontarli” — il team ha condotto un’analisi preventiva per identificare i possibili rischi del progetto FleetManager e pianificare azioni di mitigazione. L’obiettivo è ridurre l’impatto sul rispetto delle scadenze, sulla qualità e sulla funzionalità complessiva del sistema.

6.2 Classificazione dei rischi

I rischi vengono suddivisi in quattro categorie principali:

- Tecnici: problemi legati a strumenti, librerie, configurazioni o errori di integrazione tra moduli;
- Organizzativi: difficoltà di coordinamento, comunicazione o gestione dei tempi di lavoro
- Operativi: problemi legati ai test, alla qualità del codice o all’aggiornamento della documentazione;
- Di pianificazione: ritardi nella consegna delle milestone o stima errata del carico di lavoro.

6.3 Tabella di analisi dei rischi

Rischio	Categoria	Probabilità	Impatto	Azioni di mitigazione
Ritardo nell'integrazione dei moduli (es. prenotazioni, scadenze)	Tecnico	Media	Alta	Integrazione continua con test incrementali e commit frequenti.
Difficoltà nell'uso di Papyrus per la sincronizzazione UML ↔ codice	Tecnico	Media	Media	Formazione iniziale e creazione di template standard per i diagrammi.
Sovraccarico di lavoro o indisponibilità temporanea di un membro del team	Organizzativo	Media	Alta	Pianificazione bilanciata delle attività e backup reciproco tra Pietro e Francesco.
Errori o incompletezza nella documentazione	Operativo	Media	Media	Aggiornamento dei documenti a ogni sprint e revisione incrociata.
Test JUnit non sufficienti o mancata copertura minima	Operativo	Bassa	Alta	Definizione di casi di test in parallelo all'implementazione.
Problemi di compatibilità o configurazione Maven	Tecnico	Media	Media	Verifica costante del file pom.xml e uso di versioni stabili delle librerie.
Rallentamenti dovuti a strumenti o hardware personale non aggiornato	Tecnico	Bassa	Media	Backup periodico e test incrociati su più macchine.
Mancato rispetto delle milestone intermedie	Pianificazione	Media	Alta	Timeboxing rigido, verifica settimanale e aggiornamento del project board.
Incoerenza tra codice e diagrammi UML	Operativo	Media	Media	Revisione congiunta diagrammi–codice al termine di ogni sprint.

7. Personale

7.1 Struttura del team

Il progetto FleetManager è sviluppato da un team composto da due membri, che operano in modo collaborativo e integrato durante tutte le fasi del ciclo di vita del software.

7.2 Competenze richieste

Il progetto richiede competenze diversificate, sia tecniche sia organizzative. Ogni membro ha consolidato o acquisirà le seguenti abilità:

Area	Competenze richieste	Modalità di acquisizione
Analisi e Requisiti	Identificazione requisiti funzionali e non funzionali, tracciamento use case	Studio delle slide del corso, confronto interno
Progettazione UML	Use Case, Class, State, Sequence, Activity, Component, Package Diagram (Papyrus)	Esercitazioni e lavoro incrementale sul modello
Sviluppo Software	Java 17, Eclipse, Maven, DAO Pattern, Log4j2	Implementazione diretta e revisione reciproca
Testing & QA	JUnit 5, metriche McCabe, analisi statica (SonarQube)	Autovalutazione e test iterativi
Configuration Management	Uso avanzato di GitHub (branch, PR, issue, tag)	Applicazione pratica nel ciclo di sviluppo
Documentazione	Redazione Project Plan e altri 5 documenti richiesti	Aggiornamento continuo a fine sprint

7.3 Distribuzione temporale del personale

Anche se il team rimane costante, la concentrazione dell'impegno varia secondo le fasi del progetto:

Fase	Durata stimata
Analisi e Requisiti (Sprint 0–1)	~15 ore
Progettazione UML (Sprint 1–2)	~20 ore
Implementazione (Sprint 2–4)	~40 ore
Testing e QA (Sprint 4–5)	~15 ore
Documentazione e Presentazione (Sprint 5)	~10 ore
Totale stimato	100 ore ($\approx$ 50 ore/persona)

7.4 Disponibilità e coordinamento

- Meeting interni: settimanalmente per pianificazione e revisione milestone.
- Strumenti di comunicazione: GitHub (issues / PR review) + Discord per sincronizzazione rapida.
- Decisioni tecniche: prese congiuntamente.

7.5 Formazione e aggiornamento

Le competenze richieste sono in larga parte già acquisite grazie alle esercitazioni di laboratorio. Tuttavia, per garantire uniformità metodologica, il team prevede:

- Autoformazione su Papyrus UML e Maven build management;
- Verifica incrociata delle procedure di testing e qualità;
- Revisione documentale reciproca prima di ogni milestone.

7.6 Sintesi

Il personale coinvolto nel progetto FleetManager è composto da due sviluppatori che coprono l'intero ciclo di vita del software, con ruoli e competenze equilibrate. L'organizzazione è stabile, le responsabilità sono chiare e l'impegno è distribuito uniformemente. Questo assetto garantisce la continuità del lavoro e la possibilità di mantenere un elevato livello di qualità e coordinamento in tutte le fasi del progetto.

8. Metodi e tecniche

8.1 Metodi applicati lungo il ciclo di vita

Il progetto FleetManager segue un processo iterativo-incrementale con fasi successive ma parzialmente sovrapposte.

Durante ogni iterazione vengono applicati metodi specifici per ciascuna fase dell'ingegneria del software:

Fase	Metodi e strumenti principali
Analisi dei requisiti	Metodo IEEE 830 per la definizione dei requisiti funzionali e non funzionali; raccolta tramite brainstorming di gruppo e formalizzazione in Use Case Diagram (Papyrus).
Progettazione	Modellazione UML completa (Use Case, Class, Sequence, Activity, State, Component e Package Diagram); architettura basata su pattern MVC + DAO; design pattern Singleton, Observer, State e Strategy.
Implementazione	Programmazione in Java 17 con Eclipse e gestione dipendenze via Maven; codice organizzato in moduli (model, service, repository, ui, test); logging tramite Log4j2.
Testing	JUnit 5 per test di unità e integrazione; misurazione copertura con Maven Surefire; metriche di qualità (SonarQube).
Manutenzione	Refactoring progressivo e aggiornamento continuo dei diagrammi UML; gestione modifiche con branch dedicati e pull request revisionate.

8.2 Tecniche di gestione della configurazione

Il controllo di versione è gestito interamente con GitHub. Le principali convenzioni operative sono:

- Branching model:
 - main → stabile e rilasciabile;
 - develop → sviluppo attivo;
 - branch funzionale develop per nuove implementazioni;
- Pull Request con revisione reciproca obbligatoria.
- Tag semantici per le milestone (v1.0.0, v1.1.0, ...).
- Issues per la tracciabilità dei cambiamenti e dei bug.
- File .gitignore per escludere artefatti generati (/target, *.class, *.log).

La configurazione del progetto (struttura cartelle, dipendenze, plugin) è centralizzata nel file pom.xml. Ogni modifica sostanziale al build o alle librerie è documentata in changelog.md.

8.3 Ambiente di sviluppo e di prova

Componente	Strumento / versione	Ruolo
IDE	Eclipse 2024-09	sviluppo e debug
Linguaggio	Java 17	implementazione logica
Build / Gestione dipendenze	Maven 3.9+	compilazione e test
Modellazione	Papyrus 2025	diagrammi UML
Database	H2 1.4.x (embedded)	persistenza locale
Testing	JUnit 5, Maven Surefire	test automatici
QA	SonarQube	analisi statica e metriche
Version control	GitHub	collaborazione e tracking

Tutti i test vengono eseguiti su macchine personali dei due sviluppatori (Windows 10 e MacOS Tahoe), con JDK 17 e ambiente Maven configurato. L'esecuzione dei test di integrazione non richiede connessione di rete né hardware aggiuntivo.

8.4 Ordine d'integrazione e test

L'integrazione dei componenti segue un approccio bottom-up, per ridurre l'impatto di errori di dipendenza:

1. Componenti di base: classi di dominio (Veicolo, Utente, Prenotazione).
2. Repository / DAO: funzioni CRUD e gestione persistenza.
3. Service Layer: logica applicativa (prenotazioni, scadenze, manutenzioni).
4. Gestione notifiche e pattern Observer.
5. Testing JUnit completo e verifica funzionale.

8.5 Procedure di test e accettazione

Ogni test prevede:

1. Identificativo e descrizione;
2. Dati in input e output atteso;
3. Risultato effettivo (PASS / FAIL);
4. Collegamento al requisito di riferimento.

I risultati vengono raccolti nel file /documenti/testing.pdf con tabelle di esito e copertura.

8.6 Documentazione tecnica prodotta

Durante le fasi di sviluppo vengono generati automaticamente i seguenti artefatti:

- JavaDoc con `mvn javadoc:javadoc`;
- report JUnit (XML/HTML);
- report SonarQube;
- diagrammi UML aggiornati in Papyrus.

Tali documenti vengono archiviati nel repository sotto le cartelle `/target/reports/` e `/documenti/`, garantendo tracciabilità e verificabilità costante.

8.7 Sintesi

I metodi e le tecniche adottati assicurano un processo rigoroso, documentato e controllato in ogni fase. Il sistema di versionamento GitHub e gli strumenti Maven-JUnit permettono una gestione efficiente della configurazione e del testing. Come indicato da Van Vliet, “l’ordine in cui i componenti vengono integrati e testati deve essere esplicitamente definito”: FleetManager rispetta tale principio assicurando una pipeline di sviluppo progressiva, controllata e verificabile.

9. Garanzia di qualità

9.1 Obiettivo

L'obiettivo del piano di garanzia della qualità è assicurare che il software FleetManager soddisfi i requisiti di correttezza, affidabilità, manutenibilità, efficienza e chiarezza previsti nelle specifiche.

In linea con il capitolo 6 di Van Vliet, la qualità viene perseguita attraverso una combinazione di:

- procedure organizzative (revisioni, test, controllo di configurazione),
- strumenti di verifica automatica,
- metriche quantitative di valutazione del codice e dei processi.

9.2 Organizzazione della qualità

Poiché il team è composto da due membri, la garanzia della qualità è integrata nel processo stesso.

Le decisioni sulla qualità vengono prese in comune e discusse al termine di ogni sprint, sulla base dei risultati dei test e delle analisi statiche.

9.3 Procedure di garanzia

1. Code Review: ogni pull request deve essere revisionata dall'altro membro prima del merge su main.
2. Testing automatico: tutti i moduli devono essere coperti da test JUnit 5. La build Maven fallisce se i test non passano.
3. Analisi statica: uso di SonarQube per rilevare code smells, duplicazioni e violazioni di standard.
4. Controllo UML: coerenza periodica tra diagrammi Papyrus e codice Java.
5. Documentazione versionata: ogni sprint prevede l'aggiornamento dei documenti tecnici nel repository.
6. Retrospectiva di qualità: verifica finale a ogni sprint sul rispetto dei requisiti di qualità prefissati.

9.4 Requisiti di qualità

Il progetto adotta i seguenti criteri di qualità misurabili:

Attributo	Descrizione	Misura / Strumento
Correttezza	Il sistema produce risultati conformi ai requisiti.	100 % test JUnit superati.
Affidabilità	Assenza di crash, errori di stato o inconsistenze.	Test d'integrazione e stress test.
Manutenibilità	Codice modulare, leggibile e documentato.	Analisi SonarQube, JavaDoc completo.
Efficienza	Tempo di risposta e uso risorse proporzionati alla scala del problema.	Misure empiriche su casi di carico simulati.
Portabilità	Esecuzione su diversi sistemi Java SE 17.	Test su macchine differenti (Windows / Server 2022).
Coerenza documentale	Allineamento tra requisiti, design e implementazione.	Revisione documenti a fine sprint.

9.5 Metriche di valutazione

(da approfondire)

9.6 Ambiente di controllo qualità

- GitHub → verifica continua tramite workflow e log di commit.
- Maven Surefire → esecuzione automatica dei test.
- Papyrus → modello UML allineato con il codice.
- SonarQube → analisi statica e report.
- JUnit 5 → verifica automatica di regressione.

Ogni strumento contribuisce a un diverso livello di qualità (processo, codice o documentazione).

9.7 Audit e revisione

Ogni fase del progetto include una revisione interna della qualità, con checklist predefinita:

- completezza requisiti;
- consistenza UML–codice;
- percentuale test superati;
- aggiornamento documentazione.

In caso di deviazioni, viene aperta un'issue GitHub e pianificata la correzione nel successivo sprint.

9.8 Sintesi

Il sistema di garanzia della qualità di FleetManager è integrato nel flusso di sviluppo Agile e basato su controlli automatici, metriche oggettive e revisioni manuali. In questo modo, il software viene costantemente verificato per soddisfare i requisiti funzionali e di qualità dichiarati, come previsto dal capitolo 6 del Van Vliet.

10. Risorse

10.1 Obiettivo

Questa sezione elenca tutte le risorse necessarie allo sviluppo, alla verifica e alla consegna del progetto FleetManager.

Comprende:

- risorse hardware e infrastrutturali,
- strumenti software di supporto,
- e il personale impiegato durante le diverse fasi del processo di sviluppo.

10.2 Risorse hardware

Le risorse hardware richieste per il progetto sono modeste, poiché lo sviluppo e il test avvengono in locale su macchine personali dei membri del team.

Risorsa	Specifiche	Utilizzo
Laptop / PC	CPU Intel i5, 16 GB RAM, SSD 1TB	Sviluppo, test, documentazione
Storage esterno o cloud (GitHub)	Spazio repository remoto	Versionamento, backup, collaborazione

10.3 Risorse software

Le risorse software comprendono tutti gli strumenti utilizzati per progettazione, implementazione, testing e documentazione.

Categoria	Strumento / Versione	Funzione
IDE di sviluppo	Eclipse 2025-09 (Java 17)	Programmazione, debug
Gestione dipendenze / build	Maven 3.9+	Compilazione, test e packaging
Linguaggio	Java 17	Implementazione
Database embedded	H2 1.4.x	Persistenza dati locale
Testing	JUnit 5, Maven Surefire	Test unitari e report automatici
Logging	Log4j2	Monitoraggio e tracciamento eventi
Analisi statica	SonarQube	Controllo qualità e metriche
Modellazione UML	Papyrus 2025	Diagrammi e design architetturale
Versionamento e collaborazione	Git + GitHub	Controllo versione e revisione
Documentazione	Markdown + ReportLab (PDF)	Stesura e formattazione documenti
Comunicazione interna	Discord	Coordinamento del team

10.4 Risorse umane (personale)

Fase del processo	Risorse coinvolte	Attività principali
Analisi e requisiti	Pietro e Francesco (collaborazione diretta)	Identificazione e formalizzazione dei requisiti secondo lo standard IEEE 830.
Progettazione UML	Pietro e Francesco (co-modellazione in Papyrus)	Creazione, aggiornamento e validazione dei diagrammi UML (Use Case, Class, Sequence, Activity).
Implementazione	Pietro e Francesco (co-sviluppo)	Sviluppo dei moduli core, repository DAO, logica di business e configurazioni Maven.
Testing e QA	Pietro e Francesco (testing incrociato)	Esecuzione test JUnit, analisi statica (SonarLint/PMD), valutazione metriche di qualità.
Documentazione finale	Pietro e Francesco (redazione condivisa)	Refactoring finale, aggiornamento dei report di sprint e preparazione della presentazione.

10.5 Cicli di CPU e risorse di calcolo

Il progetto non richiede risorse computazionali elevate.

Le prove di stress (es. molteplici prenotazioni simultanee o generazione di log) vengono eseguite su laptop di fascia media.

Le build Maven e i test JUnit richiedono in media:

- CPU usage: 25–30 % durante la compilazione e i test;
- RAM usage: 400–600 MB per JVM e H2 database;
- Tempo medio di build: 10–20 secondi.

10.6 Gestione delle risorse

- GitHub funge da repository centralizzato e da Configuration Management System;
- Papyrus e Maven assicurano coerenza tra design e codice;
- Backup automatici garantiscono sicurezza dei dati e versioni precedenti dei file;
- L'utilizzo delle risorse viene monitorato e bilanciato durante ogni sprint, in base al carico di lavoro.

10.7 Sintesi

Le risorse necessarie per FleetManager sono interamente disponibili e sostenibili per un team di due persone. L'hardware standard, combinato con strumenti software professionali (Eclipse, Maven, Papyrus, GitHub, JUnit), consente di realizzare un processo di sviluppo completo, ripetibile e documentato.

Il dimensionamento è coerente con la scala del progetto e con le linee guida di Van Vliet, che richiedono la definizione di risorse fisiche, logiche e umane per ogni fase del processo.

11. Budget e programma

11.1 Obiettivo

Scopo di questa sezione è definire:

- la distribuzione delle risorse e del tempo tra le attività principali del progetto,
- la stima del budget teorico (in ore e valore economico simulato),
- e la programmazione temporale delle fasi di lavoro, come richiesto dal capitolo 7 del Van Vliet.

11.2 Stima del tempo complessivo

Il progetto FleetManager è pensato per una durata di circa 6 settimane, in linea con il calendario del corso e la scala del progetto didattico.

Fase	Durata stimata	Periodo indicativo	Output
Sprint 0 – Setup	1 settimana (10 ore tot.)	ottobre 2025	Repo GitHub + configurazione Maven/Papyrus
Sprint 1 – Requisiti & UML	1 settimana (15 ore)	fine ottobre/inizio novembre 2025	Documento Requisiti + Use Case/Class Diagram
Sprint 2 – Implementazione core	2 settimane (30 ore)	novembre – dicembre 2025	CRUD veicoli + gestione utenti + test unitari
Sprint 3 – Funzionalità avanzate	1 settimana (20 ore)	dicembre 2025	Prenotazioni, scadenze, Observer Pattern
Sprint 4 – QA e documentazione	1 settimana (15 ore)	dicembre 2025 – gennaio 2026	Report test, metriche + documenti finali
Totale stimato	≈ 90 ore (45 per persona)	6 settimane	Prodotto completo + presentazione finale

11.3 Budget teorico (stima economica accademica)

Anche se si tratta di un progetto didattico, si può simulare un costo basato sul valore medio di un junior developer (≈ € 25/ora).

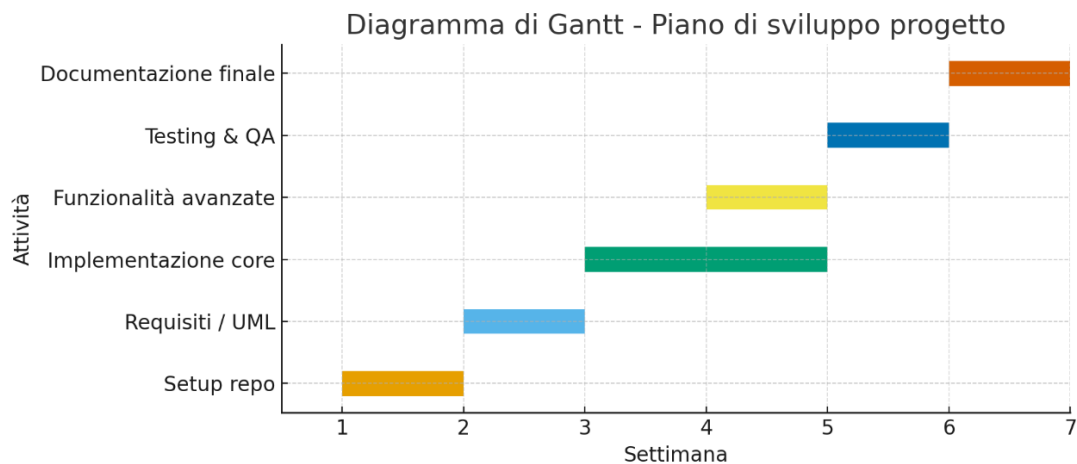
Attività	Ore totali	Costo unitario (€)	Costo stimato (€)
Analisi & Requisiti	15	25	375
Modellazione UML & Design	20	25	500
Implementazione	30	25	750
Testing & QA	15	25	375
Documentazione & Presentazione	10	25	250
Totale stimato	90	25	2 250 € (≈ 1 125 €/persona)

Questa stima serve solo a quantificare l’impegno e la ripartizione delle attività, non a rappresentare costi reali.

11.4 Allocazione risorse

Tipo di risorsa	Impiego principale	Fase coinvolta
Hardware	PC + Laptop16 GB RAM + SSD	Tutte
Software	Eclipse, Maven, Papyrus, GitHub, JUnit	Tutte
Personale	Pietro Plati (Project Manager / Backend) – Francesco Basis (QA / Modelling)	Tutte
Database H2	Archiviazione locale temporanea	Implementazione / Testing

11.5 Pianificazione temporale (diagramma Gantt semplificato)



11.6 Monitoraggio dell'avanzamento

- Le milestone vengono monitorate nel Project Board GitHub (colonne To Do / In Progress / Done).
- Gli indicatori principali sono:
 - % issue chiuse, ≥ 80 %
 - % test JUnit superati, ≥ 95 %
 - % build Maven riuscite, ≥ 95 %
 - copertura test codice, ≥ 70 %

11.7 Sintesi

Il budget di tempo e risorse di FleetManager è perfettamente coerente con la scala del progetto accademico e le linee guida del Van Vliet. La pianificazione in sprint brevi, il controllo continuo tramite GitHub e la distribuzione bilanciata delle ore consentono di monitorare costantemente i progressi e mantenere alta la qualità del prodotto finale.

12. Gestione dei cambiamenti

12.1 Obiettivo

Poiché ogni progetto software evolve nel tempo, la gestione dei cambiamenti ha lo scopo di mantenere il controllo su modifiche a requisiti, codice e documentazione, garantendo che vengano:

- analizzate,
- approvate,
- incorporate in modo coerente,

senza compromettere la qualità o la pianificazione. Il principio guida è quello espresso da Van Vliet: “I cambiamenti sono inevitabili, ma devono essere gestiti in modo ordinato attraverso procedure chiare e tracciabili”.

12.2 Tipologie di cambiamento

Nel progetto FleetManager i cambiamenti possono riguardare:

Tipo di cambiamento	Descrizione
Requisiti	Aggiunta, modifica o eliminazione di funzionalità
Progettazione	Aggiornamento diagrammi UML o architettura
Codice sorgente	Correzioni o ottimizzazioni
Documentazione	Aggiornamento dei file PDF/Markdown

12.3 Processo di gestione del cambiamento

Ogni modifica segue un flusso controllato, documentato tramite GitHub:

1. Proposta di cambiamento
 - Apertura di una GitHub Issue con titolo e descrizione chiara (“[CHANGE] Aggiunta validazione date prenotazione”).
 - Classificazione del tipo di modifica (requisito, codice, documento).
2. Valutazione dell’impatto
 - Stima qualitativa dell’impatto su:
 - Tempo (ore necessarie / eventuale ritardo),
 - Qualità (nuovi test, aggiornamento UML),
 - Documentazione (sezioni da modificare).
 - L’impatto è discusso tra Pietro Plati e Francesco Basis prima dell’approvazione.
3. Approvazione e pianificazione
 - Se la modifica è approvata, viene assegnata una priorità e collegata alla milestone corrente.
4. Implementazione controllata
 - Modifica del codice o dei diagrammi.
 - Aggiornamento immediato della documentazione coinvolta.
 - Esecuzione dei test JUnit relativi e build Maven.
5. Verifica e integrazione
 - Revisione del codice da parte dell’altro membro.
 - Merge su main solo dopo test superati e review approvata.

12.4 Criteri di approvazione

Una richiesta di modifica viene approvata solo se:

- è giustificata da un’esigenza funzionale o qualitativa;
- non compromette il completamento di milestone in corso;
- è accompagnata da test che dimostrano l’assenza di regressioni;
- mantiene coerenza tra UML – codice – documentazione.

12.5 Strumenti di supporto

- GitHub Issues → tracciamento richieste di modifica;
- Git Branch / PR → implementazione e review;
- Papyrus UML → aggiornamento grafico delle modifiche architetturali;
- Maven + JUnit → verifica automatica post-integrazione;
- Changelog.md → registro storico delle modifiche approvate.

12.6 Rischi associati ai cambiamenti

- Inserire modifiche non registrate può portare a code smell, documentazione incoerente e regressioni.
- Per evitare questi rischi, nessuna modifica diretta è ammessa sul branch main; tutto passa da PR revisionate e issue tracciate.

12.7 Incorporazione nel progetto

Dopo l'approvazione e la verifica, la modifica viene:

1. integrata nella branch principale (main),
2. testata globalmente con mvn clean test,
3. segnalata nel changelog,
4. e infine propagata alla documentazione UML e PDF.

12.8 Sintesi

La gestione dei cambiamenti in FleetManager garantisce ordine, tracciabilità e controllo. Ogni modifica è valutata, registrata, verificata e approvata prima dell'integrazione. Come raccomandato da Van Vliet, ciò consente di mantenere il progetto stabile e coerente, riducendo il rischio di superamento di tempi o degrado qualitativo.

13. Consegna

13.1 Obiettivo

La sezione di consegna definisce i deliverable finali del progetto FleetManager, le modalità di presentazione e i criteri di accettazione.

L'obiettivo è assicurare che tutti i prodotti del progetto — codice, documentazione e modelli — siano completi, coerenti e verificabili, in linea con i requisiti e con gli standard stabiliti nel piano di progetto.

13.2 Deliverable principali

Al termine del progetto, il team consegnerà i seguenti artefatti:

Tipo	Descrizione
Repository GitHub completo	Tutti i file di progetto, codice sorgente, test, documenti e diagrammi UML
Progetto Eclipse/Maven	Struttura completa e compilabile, con test automatizzati
Diagrammi UML Papyrus	Use Case, Class, State, Sequence, Activity, Component e Package Diagram
Documentazione PDF	Project Plan, Gestione progetto, Requisiti, Design, Testing, Maintenance
Testing report	Esiti dei test JUnit, coverage, metriche di qualità (SonarLint/PMD)
Presentazione finale	Slide sintetiche (max 15) per l'esame orale
Demo eseguibile	Dimostrazione funzionale del sistema (esecuzione locale)

13.3 Modalità di consegna

- La consegna avverrà tramite repository GitHub, condiviso con i docenti garganti e silviabonfanti.
- Tutti i documenti saranno inclusi nella cartella /documenti/ e nominati secondo la convenzione del corso.
- La versione finale sarà identificata con il tag v1.0.0 e sarà immutabile dopo la deadline.
- Eventuali fix successivi (per la tesi o ulteriori sviluppi) saranno pubblicati su branch post-exam.

13.4 Criteri di accettazione

Il progetto sarà considerato completo e accettabile se rispetta i seguenti criteri:

Categoria	Criterio di accettazione	Verifica
Funzionalità	Tutti i requisiti funzionali di base implementati (gestione veicoli, prenotazioni, scadenze, manutenzioni)	Test e demo
UML e Design	Tutti i diagrammi richiesti presenti, coerenti con il codice	Revisione docenti
Qualità del codice	Nessun errore compilativo, test superati, metriche rispettate	Maven + SonarLint
Documentazione	Project Plan e documenti conformi a Van Vliet	Confronto con checklist ufficiale
Versionamento	Repository organizzato, branch e tag coerenti	Verifica manuale GitHub
Presentazione	Slide chiare e complete entro 15 minuti	Discussione orale

13.5 Responsabilità di consegna

Durante la fase di consegna, Pietro Plati e Francesco Basis operano in stretta collaborazione, condividendo tutte le attività di verifica, test e preparazione dei materiali finali. Entrambi partecipano al build conclusivo del progetto, all'esecuzione dei test di validazione e al push definitivo su GitHub, assicurandosi che il repository sia completo, coerente e privo di errori. La documentazione finale e le slide di presentazione vengono redatte e revisionate congiuntamente, in modo da mantenere un allineamento costante tra codice, diagrammi UML e report tecnici.

13.6 Processo di verifica finale

(da stabilire)

13.7 Valutazione e prospettive future

Il progetto FleetManager costituisce un caso di studio completo per l'applicazione delle metodologie di Ingegneria del Software.

Al termine della consegna, il sistema potrà essere:

- esteso con un'interfaccia web o mobile,
- integrato con un database remoto o servizi REST,
- e utilizzato come base per un'eventuale tesi o progetto di ricerca.

13.8 Sintesi

La fase di consegna rappresenta la chiusura formale del progetto. Grazie alla gestione rigorosa del versionamento, alla documentazione strutturata e ai test automatizzati, FleetManager rispetta tutti i criteri di accettazione stabiliti, garantendo un prodotto software completo, verificato e conforme agli standard del corso e del modello Van Vliet.